

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 2

AIM :- A* Search and Recursive Best-First Search

1. Implement the A* Search algorithm for solving a pathfinding problem.
2. Implement the Recursive Best-First Search algorithm for the same problem.
3. Compare the performance and effectiveness of both algorithms.

Code:-

```
import heapq
import matplotlib.pyplot as plt
# 1. GRAPH NETWORK & HEURISTICS (FROM GOOGLE MAPS ROUTE DATA)
# Format: { Parent: { Child: (Distance_km, Time_minutes) } }
road_network = {
    'MVLU': {'BKC': (7, 15), 'Bandra': (6, 14)},
    # Route 1: Eastern Freeway Corridor (Fastest Time)
    'BKC': {'Freeway_Sewri': (6, 12)},
    'Freeway_Sewri': {'PD_Mello_CSMT': (12, 21)},
    'PD_Mello_CSMT': {'Gateway': (3, 5)},
    # Route 2 & 3: Western & Central Corridors
    'Bandra': {'Dadar_TT': (5, 18), 'Coastal_Rd': (6, 12)},
    # Route 2: Central Corridor (Shortest Distance)
    'Dadar_TT': {'Ambedkar_Rd': (4, 15)},
    'Ambedkar_Rd': {'PD_Mello_CSMT': (7, 12)},
    # Route 3: Coastal Rd / Marine Drive Path
    'Coastal_Rd': {'Marine_Drive': (10, 18)},
    'Marine_Drive': {'Gateway': (4, 10)},
    'Gateway': {}
}
# Heuristics: Straight-line estimates to Gateway of India
time_heuristics = {
    'MVLU': 53, 'BKC': 38, 'Bandra': 40, 'Freeway_Sewri': 26,
    'Dadar_TT': 32, 'Coastal_Rd': 28, 'Ambedkar_Rd': 17,
    'PD_Mello_CSMT': 5, 'Marine_Drive': 10, 'Gateway': 0
}
dist_heuristics = {
    'MVLU': 25, 'BKC': 18, 'Bandra': 19, 'Freeway_Sewri': 15,
    'Dadar_TT': 11, 'Coastal_Rd': 14, 'Ambedkar_Rd': 7,
    'PD_Mello_CSMT': 3, 'Marine_Drive': 4, 'Gateway': 0
}
# Spatial coordinates for clear visual separation on maps
map_coordinates = {
    'MVLU': (2.0, 6.0),
    'BKC': (3.2, 4.8),      # Shifted right (Freeway Branch)
    'Bandra': (0.8, 4.8),  # Shifted left (Central/Western Branch)
    'Freeway_Sewri': (3.5, 3.2), # Route 1: Eastern Freeway
    'Dadar_TT': (2.0, 3.6),   # Route 2: Central Corridor
    'Coastal_Rd': (0.5, 3.2), # Route 3: Western Coastal Road
    'Ambedkar_Rd': (2.0, 2.2),
    'PD_Mello_CSMT': (2.8, 1.2),
```

T101

RAJDEEPP M PARAB

TY.BSc.CS

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
'Marine_Drive': (0.8, 1.2),
'Gateway': (2.0, 0.2)
}
# 2. A* SEARCH (TIME OPTIMIZED -> ROUTE 1)
def compute_a_star_time(graph, h_map, start, goal):
    fringe = [(h_map[start], start, [start], 0, 0)] # (f, node, path, g_time, g_dist)
    settled = set()
    steps = 0
    while fringe:
        f_val, current, route, g_time, g_dist = heapq.heappop(fringe)
        steps += 1
        if current == goal:
            return route, g_time, g_dist, steps
        if current in settled:
            continue
        settled.add(current)
        for neighbor, (d_cost, t_cost) in graph[current].items():
            if neighbor not in settled:
                heapq.heappush(fringe, (g_time + t_cost + h_map[neighbor], neighbor,
                                         route + [neighbor], g_time + t_cost, g_dist + d_cost))
    return None, 0, 0, steps

# 3. RBFS (DISTANCE OPTIMIZED -> ROUTE 2)
def compute_rbfs_distance(graph, h_map, start, goal):
    def evaluate(node, current_d, current_t, f_limit, path, metrics):
        metrics['steps'] += 1
        if node == goal:
            return True, path, current_d, current_t, current_d
        neighbors = graph[node]
        if not neighbors:
            return False, [], 0, 0, float('inf')
        successors = []
        for adjacent, (d_cost, t_cost) in neighbors.items():
            if adjacent not in path:
                next_d = current_d + d_cost
                next_t = current_t + t_cost
                next_f = max(next_d + h_map[adjacent], current_d + h_map[node])
                successors.append([next_f, adjacent, next_d, next_t])
        if not successors:
            return False, [], 0, 0, float('inf')
        while True:
            successors.sort(key=lambda x: x[0])
            best = successors[0]
            if best[0] > f_limit:
                return False, [], 0, 0, best[0]
            alt_f = successors[1][0] if len(successors) > 1 else float('inf')
            success, final_p, final_d, final_t, back_f = evaluate(
                best[1], best[2], best[3], min(f_limit, alt_f), path + [best[1]], metrics
            )
            best[0] = back_f
            if success:
                return True, final_p, final_d, final_t, final_d
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
        return True, final_p, final_d, final_t, back_f
run_metrics = {'steps': 0}
_, route, d_cost, t_cost, _ = evaluate(start, 0, 0, float('inf'), [start], run_metrics)
return route, d_cost, t_cost, run_metrics['steps']
```

4. VISUALIZATION FUNCTIONS

```
def draw_map_graph(title, graph, h_map, coords, active_path, total_time, total_dist, highlight_color):
    plt.figure(figsize=(10, 7.5))
    # 1. Render Path Edges
    for location, connections in graph.items():
        x1, y1 = coords[location]
        for target, (dist, time) in connections.items():
            x2, y2 = coords[target]
            on_path = (location in active_path and target in active_path and
                       active_path.index(target) == active_path.index(location) + 1)
            line_color = highlight_color if on_path else '#dfe4ea'
            line_thick = 5.5 if on_path else 1.5
            z_order = 2 if on_path else 1
            plt.plot([x1, x2], [y1, y2], color=line_color, linewidth=line_thick, zorder=z_order)
            label_text = f"{dist}km / {time}m"
            plt.text((x1 + x2) / 2, (y1 + y2) / 2, label_text,
                     color='#2c3e50' if on_path else '#a4b0be',
                     fontsize=8, fontweight='bold' if on_path else 'normal', ha='center', va='center',
                     bbox=dict(facecolor='white', alpha=0.9, edgecolor='none', pad=2.0))
    for location, (x, y) in coords.items():
        is_active = location in active_path
        node_color = highlight_color if is_active else '#747d8c'
        scale = 1300 if is_active else 800
        plt.scatter(x, y, color=node_color, s=scale, zorder=3, edgecolor='#2c3e50', linewidth=1.2)
        plt.text(x, y, f"{location}\nh={h_map[location]}", ha='center', va='center',
                 color='white', fontsize=7.5, fontweight='bold')
    plt.title(f"{title}\nPath Metrics: {total_dist} km | {total_time} mins", fontsize=11, fontweight='bold')
    plt.axis('off')
    plt.tight_layout()
def draw_comparison_chart(a_path_len, a_steps, r_path_len, r_steps):
    plt.figure(figsize=(8, 5))
    categories = ['A* Search (Time Opt)', 'RBFS (Distance Opt)']
    x = [0, 1]
    width = 0.25
    bars_path = plt.bar([pos - width/2 for pos in x], [a_path_len, r_path_len], width=width, label='Nodes in Path', color='#2ecc71', edgecolor='black')
    bars_steps = plt.bar([pos + width/2 for pos in x], [a_steps, r_steps], width=width, label='Search Steps Taken', color='#9b59b6', edgecolor='black')
    plt.xticks(x, categories, fontweight='bold')
    plt.ylabel('Metric Count Value', fontsize=10, fontweight='bold')
    plt.title('Performance Metric Comparison: Final Path Nodes vs Memory Steps', fontsize=11, fontweight='bold')
    plt.grid(axis='y', linestyle='--', alpha=0.3)
    plt.legend(loc='upper left')
    for bar in bars_path + bars_steps:
        yval = bar.get_height()
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

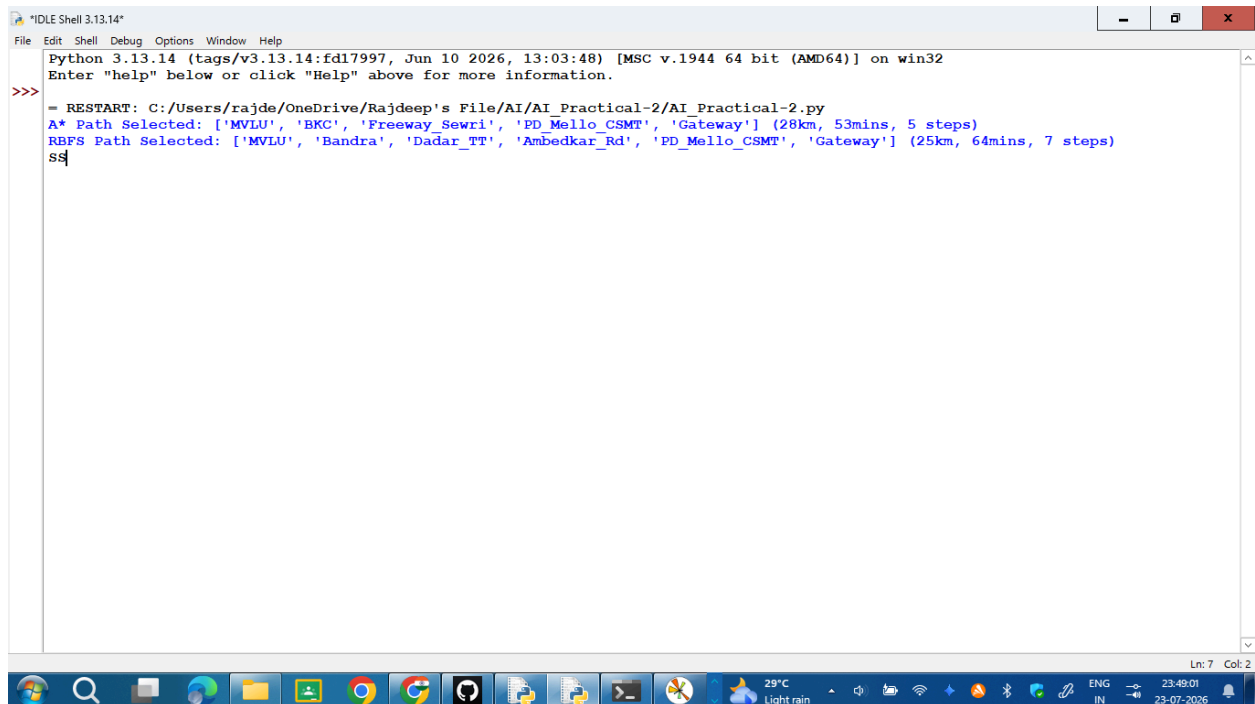
SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
plt.text(bar.get_x() + bar.get_width()/2.0, yval + 0.1, str(yval), ha='center', va='bottom',
fontweight='bold', fontsize=9)
plt.tight_layout()
```

5. EXECUTION

```
if __name__ == "__main__":
    start_node, end_node = 'MVLU', 'Gateway'
    a_route, a_time, a_dist, a_steps = compute_a_star_time(road_network, time_heuristics, start_node,
end_node)
    r_route, r_dist, r_time, r_steps = compute_rbfs_distance(road_network, dist_heuristics, start_node,
end_node)
    print(f"A* Path Selected: {a_route} ({a_dist}km, {a_time}mins, {a_steps} steps)")
    print(f"RBFS Path Selected: {r_route} ({r_dist}km, {r_time}mins, {r_steps} steps)")
    # Graph 1: A* Path
    draw_map_graph("Graph 1: A* Search Path (Optimized for Travel Time via Freeway)",
road_network, time_heuristics, map_coordinates, a_route, a_time, a_dist, '#2ecc71')
    # Graph 2: RBFS Path
    draw_map_graph("Graph 2: RBFS Search Path (Optimized for Distance via Dadar TT)",
road_network, dist_heuristics, map_coordinates, r_route, r_time, r_dist, '#9b59b6')
    # Graph 3: Comparison Bar Chart
    draw_comparison_chart(len(a_route), a_steps, len(r_route), r_steps)
    plt.show()
```

Output:-

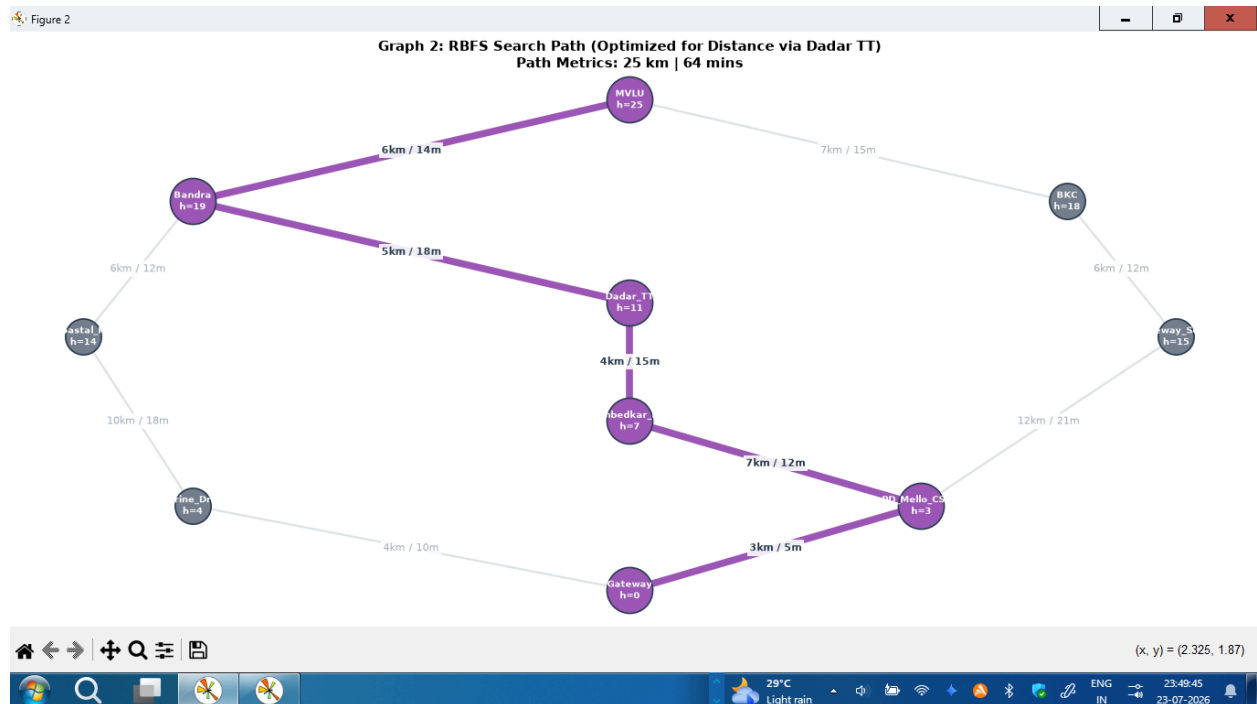
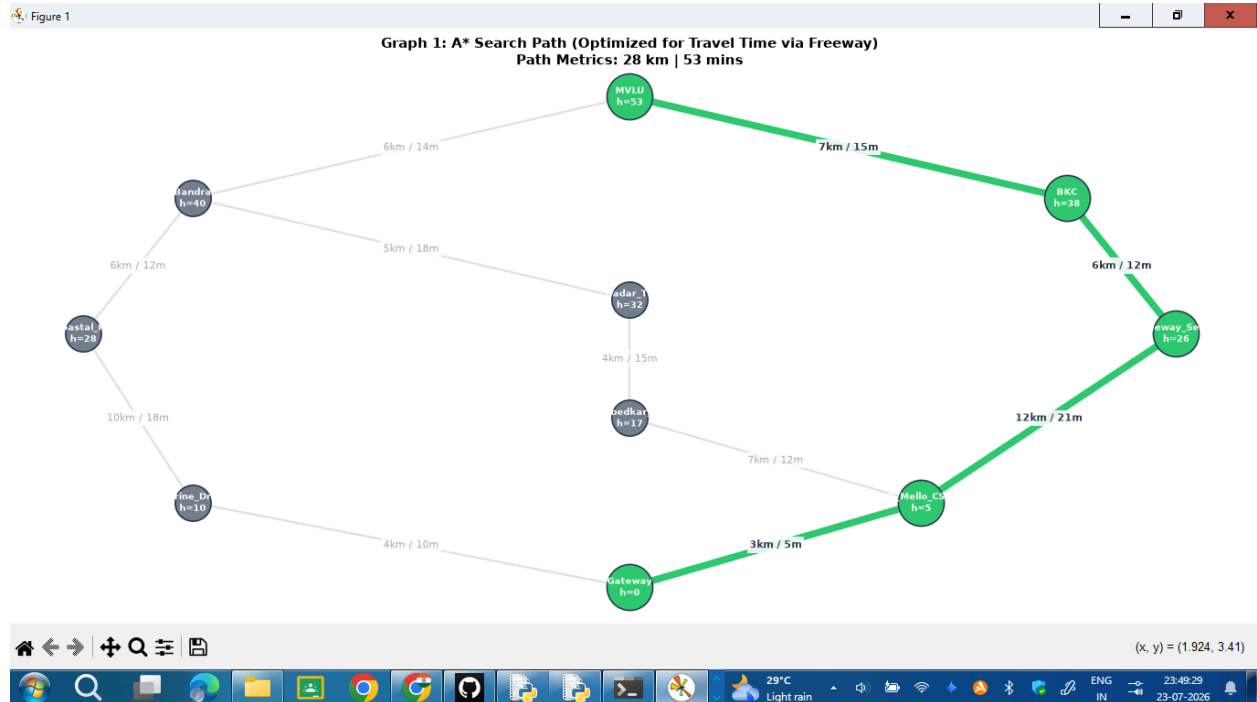


```
*IDLE Shell 3.13.14*
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI Practical-2/AI Practical-2.py
A* Path Selected: ['MVLU', 'BKC', 'Freeway Sewri', 'PD Mello CSMT', 'Gateway'] (28km, 53mins, 5 steps)
RBFS Path Selected: ['MVLU', 'Bandra', 'Dadar TT', 'Ambedkar Rd', 'PD Mello CSMT', 'Gateway'] (25km, 64mins, 7 steps)
ss
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical



SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

